

Relatório de Engenharia Reversa

- GRIS Login Super Seguro v1

O objetivo desse chall é descobrir a senha que abre a aplicação contendo a flag, por meio de engenharia reversa sobre o executável passado. Fazendo uma análise estática sobre o programa pelo Ghidra, obtemos um código decompilado que contém basicamente uma comparação com um valor de senha para que a função de entrega da flag seja chamada, como pode ser visto na imagem abaixo:

```
Listing: login-ultra-secreto-1.elf - (12 addresses selected)
00101392 48 8d 05 LEA RAX,[DAT_001020a4]
00101399 48 89 c7 MOV RDI=>DAT_001020a4,RAX
0010139c b8 00 00 MOV EAX,0x0
001013a1 e8 8a fd CALL <EXTERNAL>::printf
001013a6 48 8d 45 f4 LEA RAX=>local_14,[RBP + -0xc]
001013aa 48 89 c6 MOV RSI,RAX
001013ad 48 8d 05 LEA RAX,[DAT_001020a7]
001013b4 48 89 c7 MOV RDI=>DAT_001020a7,RAX
001013b7 b8 00 00 MOV EAX,0x0
001013bc e8 af fd CALL <EXTERNAL>::__isoc99_scanf
001013c1 8b 45 f4 MOV EAX,dword ptr [RBP + local_14]
001013c4 3d 39 05 CMP EAX,0x539
001013c9 75 0c JNZ LAB_001013d7
001013cb b8 00 00 MOV EAX,0x0
001013d0 e8 94 fe CALL flag
001013d5 eb 0f JMP LAB_001013e6
LAB_001013d7 48 8d 05 LEA RAX,[s_Senha_incorreta!_001020aa]
001013de 48 89 c7 MOV RDI=>s_Senha_incorreta!_001020aa,RAX
001013e1 e8 0a fd CALL <EXTERNAL>::puts
LAB_001013e6
XREF[1]: 0010
```

```
Decompile: main - (login-ultra-secreto-1.elf)
1
2 undefined8 main(void)
3
4 {
5     long in_FS_OFFSET;
6     int local_14;
7     long local_10;
8
9     local_10 = *(long *)(&in_FS_OFFSET + 0x28);
10    setvbuf(stdout,(char *)0x0,2,0);
11    puts("=====");
12    puts("GRIS Login Ultra Seguro v2.14");
13    puts("=====");
14    printf("%s","Insira a senha: ");
15    __isoc99_scanf(&DAT_001020a7,&local_14);
16    if (local_14 == 0x539) {
17        flag();
18    }
19    else {
20        puts("Senha incorreta!");
21    }
22    if (local_10 != *(long *)(&in_FS_OFFSET + 0x28)) {
23        /* WARNING: Subroutine does not return */
24        __stack_chk_fail();
25    }
26    return 0;
27 }
28
```

É possível notar que a linha 15 no código decompilado é o receptor do input do usuário, e seu valor é armazenado no espaço da memória aí chamado de “&local_14”. Após a coleta do input, é feita uma comparação com o valor da senha, que embora possa parecer um registrador em um primeiro momento, na verdade é um número inteiro em sua forma hexadecimal, como pode ser visto na conversão do próprio Ghidra:

```
Listing: login-ultra-secreto-1.elf
00101374 e8 77 fd CALL <EXTERNAL>::puts
00101379 48 8d 05 LEA RAX,[s_-----_00102057]
00101380 47 0c 00 MOV RDI=>s_-----_00102057,...
00101383 e8 68 fd CALL <EXTERNAL>::puts
00101388 48 8d 05 LEA RAX,[s_Insira_a_senha:_00102093]
0010138f 48 89 c6 MOV RSI=>s_Insira_a_senha:_00102093,RAX
00101392 48 8d 05 LEA RAX,[DAT_001020a4]
00101399 48 89 c7 MOV RDI=>DAT_001020a4,RAX
0010139c b8 00 00 MOV EAX,0x0
001013a1 e8 8a fd CALL <EXTERNAL>::printf
001013a6 48 8d 45 f4 LEA RAX=>local_14,[RBP + -0xc]
001013aa 48 89 c6 MOV RSI,RAX
001013ad 48 8d 05 LEA RAX,[DAT_001020a7]
001013b4 48 89 c7 MOV RDI=>DAT_001020a7,RAX
001013b7 b8 00 00 MOV EAX,0x0
001013bc e8 af fd CALL <EXTERNAL>::__isoc99_scanf
001013c1 8b 45 f4 MOV EAX,dword ptr [RBP + local_14]
001013c4 3d 39 05 CMP EAX,0x539
001013c9 75 0c JNZ LAB_001013d7
001013cb b8 00 00 MOV EAX,0x0
001013d0 e8 94 fe CALL flag
001013d5 eb 0f JMP LAB_001013e6
LAB_001013d7 48 8d 05 LEA RAX,[s_Senha_incorreta!_001020aa]
001013de 48 89 c7 MOV RDI=>s_Senha_incorreta!_001020aa,RAX
001013e1 e8 0a fd CALL <EXTERNAL>::puts
LAB_001013e6
XREF[1]: 0010
```

```
Decompile: main - (login-ultra-secreto-1.elf)
1
2 undefined8 main(void)
3
4 {
5     long in_FS_OFFSET;
6     int local_14;
7     long local_10;
8
9     local_10 = *(long *)(&in_FS_OFFSET + 0x28);
10    setvbuf(stdout,(char *)0x0,2,0);
11    puts("=====");
12    puts("GRIS Login Ultra Seguro v2.14");
13    puts("=====");
14    printf("%s","Insira a senha: ");
15    __isoc99_scanf(&DAT_001020a7,&local_14);
16    if (local_14 == 0x539) {
17        flag();
18    }
19    else {
20        puts("Senha incorreta!");
21    }
22    if (local_10 != *(long *)(&in_FS_OFFSET + 0x28)) {
23        /* WARNING: Subroutine does not return */
24        __stack_chk_fail();
25    }
26    return 0;
27 }
28
```

Rename Variable		L
Retype Variable		Ctrl+L
Commit Params/Return		P
Commit Local Names		
Highlight		>
Secondary Highlight		>
Remove Convert/Equate		
Set Equate...		E
Binary:	...	0b0000010100111001
Decimal:	...	1337
Float:	...	1.8735360E-42
Double:	...	6.605657684897466E-321

Dessa forma, a senha da aplicação é 1337, e a sua digitação entrega a flag: [CTF-BR{s3nh@-tr@nqu1l1nh@...}](#)

- GRIS Login Super Seguro v2

Assim como na versão v1, o objetivo desse chall é encontrar a senha que passe pela aplicação, que irá fornecer ao fim a flag. Fazendo uma análise estática no Ghidra sobre o código fornecido, observa-se no código decompilado que a senha é lida do input do usuário, em seguida é encaminhada para uma função chamada “senha_check”, que retorna 0 ou 1 dependendo se a senha foi ou não correta. É possível notar que se a senha for correta, a flag é fornecida como resposta. A imagem abaixo mostra a função “main” do executável, que chama a função “senha_check”

```
Listing: login-ultra-secreto-2.elf
0010142c 48 89 c6 MOV RSI=>s_Inpira_a_senha:_00102093,RAX
0010142f 48 bd 05 LEA RAX,[DAT_001020a4]
00101436 48 89 c7 MOV RDI=>DAT_001020a4,RAX
00101439 b8 00 00 MOV EAX,0x0
0010143e e8 ed fc CALL <EXTERNAL>::printf
00101443 48 bb 15 MOV RDX,qword ptr [stdIn]
0010144a 48 bd 95 LEA RAX=>local_d8,[RBP + -0xd0]
00101451 be c8 00 MOV ESI,0xc8
00101456 48 89 c7 MOV RDI,RAX
00101459 e8 e2 fc CALL <EXTERNAL>::fgets
0010145e 48 bd 95 LEA RAX=>local_d8,[RBP + -0xd0]
00101465 48 89 c7 MOV RDI,RAX
00101468 e8 b5 fe CALL senha_check
0010146d ff ff TEST EAX,EAX
0010146f 74 0c JZ LAB_0010147d
00101471 b8 00 00 MOV EAX,0x0
00101476 e8 ee fd CALL flag
0010147b ff ff JMP LAB_0010148c
0010147d 48 bd 05 LEA RAX,[s_Senha_incorreta!_001020a7]
00101484 48 89 c7 MOV RDI=>s_Senha_incorreta!_001020a7,RAX

Decompile: main - (login-ultra-secreto-2.elf)
1
2 undefined8 main(void)
3
4 {
5     int iVar1;
6     long in_FS_OFFSET;
7     char local_d8 [200];
8     long local_10;
9
10    local_10 = *(long *) (in_FS_OFFSET + 0x28);
11    setvbuf(stdout,(char *)0x0,2,0);
12    puts("Insira a senha:");
13    puts("GRIS Login Ultra Seguro v2.14");
14    printf("Insira a senha: ");
15    fgets(local_d8,200,stdin);
16    iVar1 = senha_check(local_d8);
17    if (iVar1 == 0) {
18        puts("Senha incorreta!");
19    }
20    else {
21        flag();
22    }
23    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
24        /* WARNING: Subroutine does not return */
25        __stack_chk_fail();
26    }
27    return 0;
28 }
29
30
```

Analisando a função “senha_check”, verifica-se que ela basicamente realiza um loop de while que compara cada caractere da string “DaavizinReiDelas” com a fornecida pelo usuário, que entra na função como “param_1”. Caso exista qualquer caractere que seja diferente nessa comparação, a função imediatamente retorna 0 (false), enquanto caso todos sejam iguais retorna 1 (true).

```
Listing: login-ultra-secreto-2.elf
00101322 f3 0f 1e fa ENDBR64
00101326 55 PUSH RBP
00101327 48 89 e5 MOV RBP,RSP
0010132a 48 83 ec SUB RSP,0x40
0010132e 48 89 7d c8 MOV qword ptr [RBP + local_40],RDI
00101332 64 4b 8b MOV RAX,qword ptr FS:[0x28]
0010133b 48 89 45 f8 MOV qword ptr [RBP + local_10],RAX
0010133f 31 c0 XOR EAX,EAX
00101341 4b b8 44 MOV RAX,0x6e697a6976616144
0010134b 4b b8 52 MOV RDX,0x73616c6544696552
00101355 48 89 45 e0 MOV qword ptr [RBP + local_28],RAX
00101359 48 89 55 e8 MOV qword ptr [RBP + local_20],RDX
0010135d c6 45 f0 MOV byte ptr [RBP + local_18],0x0
00101361 c7 45 dc MOV dword ptr [RBP + local_2c],0x0
00101368 eb 29 JMP LAB_00101393
0010136a 8b 45 dc MOV EAX,dword ptr [RBP + local_2c]
0010136d 4b 90 CODE
0010136f 0f b6 54 MOVZX EDI,byte ptr [RBP + RAX*0x1 + -0x20]
00101374 8b 45 dc MOV EAX,dword ptr [RBP + local_2c]
00101377 4b 63 c8 MOVSCD RCX,EAX
0010137a 4b 8b 45 c8 MOV RAX,qword ptr [RBP + local_40]
0010137e 4b 01 c8 ADD RAX,RCX
00101381 0f b6 00 MOVZX EAX,byte ptr [RAX]
00101384 38 c2 CMP DL,AL
00101386 74 07 JAR 0010138f

Decompile: senha_check - (login-ultra-secreto-2.elf)
1
2 undefined8 senha_check(long param_1)
3
4 {
5     undefined8 uVar1;
6     long in_FS_OFFSET;
7     int local_2c;
8     char local_28 [24];
9     long local_10;
10
11    local_10 = *(long *) (in_FS_OFFSET + 0x28);
12    builtin_strncmp(local_28,"DaavizinReiDelas",0x11);
13    local_2c = 0;
14    do {
15        if (local_28[local_2c] == '\0') {
16            uVar1 = 1;
17            LAB_001013a6:
18            if (*(long *) (in_FS_OFFSET + 0x28) != *(long *) (in_FS_OFFSET + 0x28)) {
19                /* WARNING: Subroutine does not return */
20                __stack_chk_fail();
21            }
22            return uVar1;
23        }
24        if (local_28[local_2c] != (char *) (param_1 + local_2c)) {
25            uVar1 = 0;
26            goto LAB_001013a6;
27        }
28        local_2c = local_2c + 1;
29    } while( true );
30
31}
```

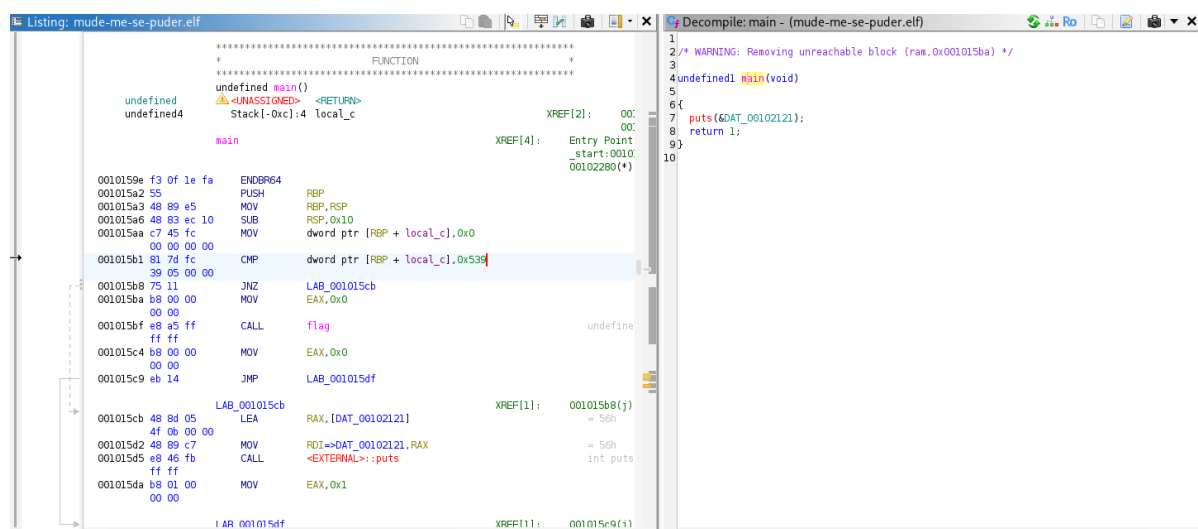
Podemos, então, concluir que a senha da aplicação é “DaavizinReiDelas”. Colocando-a no input da aplicação no servidor, recebemos a flag:

CTF-BR{0k-sTR1ngs-3m-PL@1nT3xt-N@0-s40-um@-b0a-ld3IA}

- Mude-me, se puder!

Começando por uma análise estática, observa-se inicialmente que o código em desassembly da função main não contém nada útil. Parece que ela foi escrita de um modo que o desassembly não consegue ler, e entrar nas chamadas feitas dentro dela levam a outras funções também meio vazias ou que dificilmente se consegue entender.

Porém, analisando o código em assembly, verificam-se algumas lógicas interessantes: aparentemente, a função “flag” é chamada quando a comparação da variável [RBP + local_c] com o valor hexadecimal 0x539 (1337, em decimal) é diferente de zero (logo, quando é true), assim podemos pressupor que a senha é 1337. No entanto, não há espaço para digitação de input, então de fato a grande questão desse chall é conseguir mudar essa variável no decorrer da execução do código para chamar a função flag, como o próprio nome do chall diz. Imagem da função “main” em desassembly e em assembly abaixo:



Sendo assim, troquei a forma de análise e me voltei à análise dinâmica, que serve para analisar e mudar variáveis no meio da execução do código. Para isso, utilizei o gdb.

Ao utilizar o comando de disassembly da função “main”, verifiquei exatamente o comportamento já analisado na análise estática: a função “flag” é chamada apenas se o valor do registrador [%rbp – 0x4] for igual a 1337. Dessa maneira, para chegarmos à flag, era preciso usar a funcionalidade de set de variáveis do gdb.

```
(gdb) disas main
Dump of assembler code for function main:
0x000000000000159e <+0>:    endbr64
0x00000000000015a2 <+4>:    push    %rbp
0x00000000000015a3 <+5>:    mov     %rsp,%rbp
0x00000000000015a6 <+8>:    sub     $0x10,%rsp
0x00000000000015aa <+12>:   movl    $0x0,-0x4(%rbp)
0x00000000000015b1 <+19>:   cmpl    $0x539,-0x4(%rbp)
0x00000000000015b8 <+26>:   jne     0x15cb <main+45>
0x00000000000015ba <+28>:   mov     $0x0,%eax
0x00000000000015bf <+33>:   call    0x1569 <flag>
0x00000000000015c4 <+38>:   mov     $0x0,%eax
0x00000000000015c9 <+43>:   jmp     0x15df <main+65>
0x00000000000015cb <+45>:   lea     0xb4f(%rip),%rax
0x00000000000015d2 <+52>:   mov     %rax,%rdi
0x00000000000015d5 <+55>:   call    0x1120 <puts@plt>
0x00000000000015da <+60>:   mov     $0x1,%eax
0x00000000000015df <+65>:   leave
0x00000000000015e0 <+66>:   ret
End of assembler dump.
(gdb) █
```

Para isso, primeiramente coloquei um breakpoint na instrução de comparação. Em seguida, botei o código para rodar com. Quando o código parou no breakpoint colocado, setei o valor do registrador [%rbp - 0x4] para 1337. Fazendo o código continuar sua execução, a função flag foi chamada, e com isso consegui a flag. Imagem do terminal usando gdb a seguir:

```
(gui@kalibon)-[~/Área de trabalho/GRIS/Eng Reversa]
$ gdb -q ./mude-me-se-puder.elf

Reading symbols from ./mude-me-se-puder.elf...
(No debugging symbols found in ./mude-me-se-puder.elf)
(gdb) desas main
Undefined command: "desas". Try "help".
(gdb) disas main
Dump of assembler code for function main:
   0x000000000000159e <+0>:    endbr64
   0x00000000000015a2 <+4>:    push    %rbp
   0x00000000000015a3 <+5>:    mov     %rsp,%rbp
   0x00000000000015a6 <+8>:    sub     $0x10,%rsp
   0x00000000000015aa <+12>:   movl    $0x0,-0x4(%rbp)
   0x00000000000015b1 <+19>:   cmpl    $0x539,-0x4(%rbp)
   0x00000000000015b8 <+26>:   jne     0x15cb <main+45>
   0x00000000000015ba <+28>:   mov     $0x0,%eax
   0x00000000000015bf <+33>:   call    0x1569 <flag>
   0x00000000000015c4 <+38>:   mov     $0x0,%eax
   0x00000000000015c9 <+43>:   jmp     0x15df <main+65>
   0x00000000000015cb <+45>:   lea     0xb4f(%rip),%rax          # 0x2121
   0x00000000000015d2 <+52>:   mov     %rax,%rdi
   0x00000000000015d5 <+55>:   call    0x1120 <puts@plt>
   0x00000000000015da <+60>:   mov     $0x1,%eax
   0x00000000000015df <+65>:   leave
   0x00000000000015e0 <+66>:   ret
End of assembler dump.
(gdb) break *0x5555555555b1
Breakpoint 1 at 0x5555555555b1
(gdb) run
Starting program: /home/gui/Área de trabalho/GRIS/Eng Reversa/mude-me-se-puder.elf
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x00005555555555b1 in main ()
(gdb) set *((int*)(%rbp - 0x4)) = 1337
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) n
Program not restarted.
(gdb) continue
Continuing.
CTF-BR{c0m0_mud@R_3M_runt1m3?}
[Inferior 1 (process 13145) exited normally]
(gdb) █
```

Flag: CTF-BR{c0m0_mud@R_3M_runt1m3?}

